

Week 1: Introduction to Databases

Mehnaz Khan

Today's Schedule

- Attendance
- Course Intro
- Course Contents, Syllabus
- Assessments
- Brief Intro about databases (PowerPoint Slides)
- Lunch break
- Info about Project Work

Overview

- Basic Definitions
- Database Applications
- Types of Databases
- File System vs DBMS
- Advantages of Using the DBMS
- Database Users

Basic Concepts

- **Data:** Raw facts that can be recorded (e.g., numbers, names, dates).
Have meaning in a given context.
- **Database:** An organized collection of related data.
Represents aspects of the real world (e.g., university, company).
- In relational databases, this data is stored in **tables** (rows & columns).
- In other models (can be as documents, graphs).

Basic Concepts

- **Entities:** Real-world objects we store data about (e.g., *Student*, *Course*).
- **Relationships:** Associations between entities (e.g., *John ENROLLED in Database course*).
- **Database Management System (DBMS):** Software to create, store, update, and query databases.

Provides **security, consistency, and controlled access**.

Examples: MySQL, PostgreSQL, Oracle, SQL Server.

Basic Concepts

- **Metadata:** “*Data about data*” → describes the structure and properties of the database.
- Stored by the DBMS in a **catalog / data dictionary**.
- Examples:
- *Table name:* **Students**
- *Column:* **Email (VARCHAR)**
- *Constraint:* **StudentID must be unique**
- **Example:** a student table’s metadata includes that *StudentID* must be unique, *Email* is a text field, etc.

Basic Concepts

- **Manipulating a Database**
- **Querying** → retrieve specific information (e.g., list all students in a course).
- **Updating** → insert, delete, modify records. e.g., Add/delete or update student information.
- **Reporting** → generate summaries and insights (e.g., average grades, monthly sales).

SQL Commands: DDL vs DML

- **DDL – Data Definition Language**
- Defines **structure** of the database (schema).
- Commands:
 - CREATE → make new tables, databases
 - ALTER → change table (add/remove columns)
 - DROP → delete tables or databases

- **DML – Data Manipulation Language**
- Deals with the **data inside** the tables.
- Commands:
 - INSERT → add new records
 - UPDATE → modify existing records
 - DELETE → remove records
 - SELECT → query/retrieve data

Quick example

- DDL
- CREATE TABLE Students (ID INT PRIMARY KEY, Name VARCHAR(50));
- DML
- INSERT INTO Students VALUES (1, 'John');
- SELECT * FROM Students;

Database Applications

- Databases are the foundation of modern applications — from banking to social media. Every organization depends on databases
- **Banking** – store customer info & track all transactions (e.g., ATM withdrawals, online transfers).
- **Airlines** – manage reservations, flight schedules, seat availability.
- **Universities** – handle student registrations, course enrollments, grades.
- **Sales & E-commerce** – track customers, products, purchases (e.g., Amazon order history).
- **Social Media** – store profiles, posts, likes, friend connections.

File System vs DBMS

- Data was initially stored and processed using file-based approach.
- **File system:** Collection of individual files accessed by application programs.
- Example: payroll stored in one file, student records in another.
- What do you think is the problem with this approach?

Disadvantages of File Systems

- **Separated & Isolated Data** – info spread across different files, hard to combine.
- **Data Duplication** – same data repeated in many places (e.g., customer address in multiple files).
- **Incompatible Files** – different formats/programs make sharing difficult.
- **Limited Security** – no central control over who accesses what.
- **Poor Scalability** – becomes messy as data grows.
- **DBMS** solves these issues by centralizing data, enforcing consistency, and supporting secure multi-user access.

Purpose of Databases in Information Systems

- **Core Purposes:**
- **Reliable Storage (Persistence)** – data is saved permanently, not lost when the system shuts down.
- **Data Integrity & Constraints** – ensures data is valid and consistent (e.g., no negative ages, unique IDs).
- **Concurrency & Transactions** – multiple users can access/update data at the same time safely.
- **Security** – authentication (who you are), authorization (what you can do), and auditing (who did what).
- **Efficient Querying & Reporting** – SQL enables fast retrieval and decision-making.
- **Backup & Recovery** – protect against data loss due to crashes or failures.

Purpose of Databases in Information Systems

- **Controlling Redundancy:** Redundancy means storing same data repeatedly at different places.
- It wastes storage space and data becomes inconsistent.
- This may happen because an update is applied to some of the files but not to others.

Types of Databases

1. Relational Databases (RDBMS)

- Data stored in **tables (rows & columns)**.
- Use **SQL** for querying.
- Examples: MySQL, PostgreSQL, Oracle.
- *Used in banking, university systems.*

Types of Databases

2. NoSQL (Not Only SQL) Databases – non-relational

- Flexible, schema-less, handle **big & unstructured data**.
- Data not stored in tables
- Data can be stored as: documents, graphs, key-value pairs
- *Used in social media, recommendation systems*
- *Will be discussed more later.*

Example of a Database

- Construct UNIVERSITY database
 - Store data to represent each student, course, section, grade report, and prerequisite as a record.
 - Relationships among the records
- Manipulation involves querying and updating

Example of a Database

- **Mini-world for the example:** Part of a UNIVERSITY environment.
- **Some mini-world *entities*:**
 - STUDENTs
 - COURSEs
 - SECTIONs (of COURSEs)
 - (academic) DEPARTMENTs
 - INSTRUCTORs

Example of a Database

- **Some mini-world *relationships*:**
 - SECTIONs *are of* specific COURSEs
 - STUDENTs *take* SECTIONs
 - COURSEs *have* prerequisite COURSEs
 - INSTRUCTORs *teach* SECTIONs
 - COURSEs *are offered by* DEPARTMENTs
 - STUDENTs *major in* DEPARTMENTs

Note: The above could be expressed in the *ENTITY-RELATIONSHIP* data model.

Example

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

Example

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
....		
....		
....		
Prerequisite_number	XXXXNNNN	PREREQUISITE

Specify structure of records
of each file by specifying **data type**
for each **data element**

Example

- **Examples of queries:**

- Retrieve the transcript
- List the names of students who took the section of the 'Database' course offered in fall 2008 and their grades in that section
- List the prerequisites of the 'Database' course

- **Examples of updates:**

- Enter a grade of 'A' for 'Smith' in the 'Database' section of last semester

Database Users

- Database users can be broadly grouped into **two categories**:
- **Actors on the Scene** (directly interact with the database)
- **Workers Behind the Scene** (don't use the data directly, but make DBMS possible)

Actors on the scene

1. Database administrators: Control and monitor database use.

- Responsibilities:
- Authorize access (who can read/write).
- Set up backups and recovery.
- Ensure security.

2. Database Designers

- Decide **what data will be stored** and **how it will be organized**.
- Responsibilities:
 - Define tables, relationships, constraints.
 - Design the ER model and schema.
 - Ensure the database supports business requirements.

Actors on the scene

3. End-Users - The largest group — people who actually **use the data**.

- For example:
 - **Naïve users** → use standard applications (e.g., bank clerk).
 - **Sophisticated users** → Sophisticated users like engineers or analysts interact directly with the database using SQL queries.

Workers Behind the Scene

- **DBMS system developers** → build the database engine itself.
- **Tool developers** → build utilities like MySQL Workbench
- **Computer system operators** → manage servers, hardware, OS where DB runs.
- **Maintenance staff** → update, optimize DBMS software.